

Enhancing Smart Contract Security Through DevSecOps: An Adaptive Approach for Vulnerability Detection

Presented By: Ashiful Alam Shuvo

Student ID: 2616179050

Course Information: CSE 534: Software Quality Assurance

Section: 01

Presented To:

Safat Siddiqui

safat.siddiqui@northsouth.edu

What is Smart Contracts ?

—

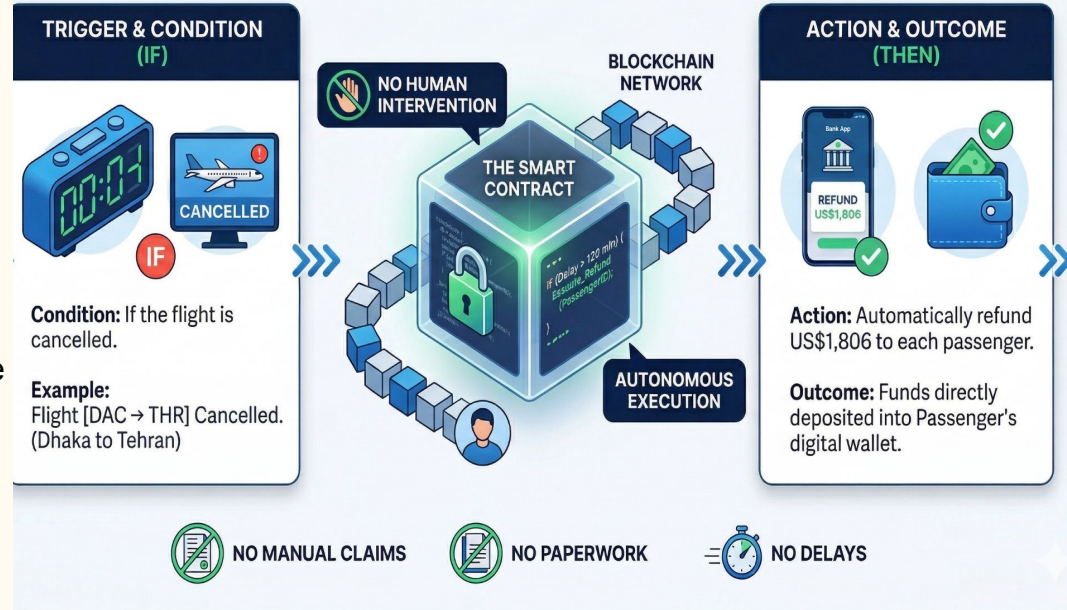
What is it?

- A "Digital Agreement" that executes itself.
- The code triggers automatically when a specific condition is met. (If this happens -> Then do that).

Why is it "Smart"?

- It is **Transparent**: Everyone can see the rules.
- It is **Immutable**: Once written, the rules cannot be changed or cheated.
- It is **Autonomous**: Works 24/7 without a middleman.

HOW A SMART CONTRACT WORKS: A PRACTICAL EXAMPLE



Why Smart Contracts Need Secure SQA ?

—

Integrating Security into SQA: The Vital Importance for Smart Contracts

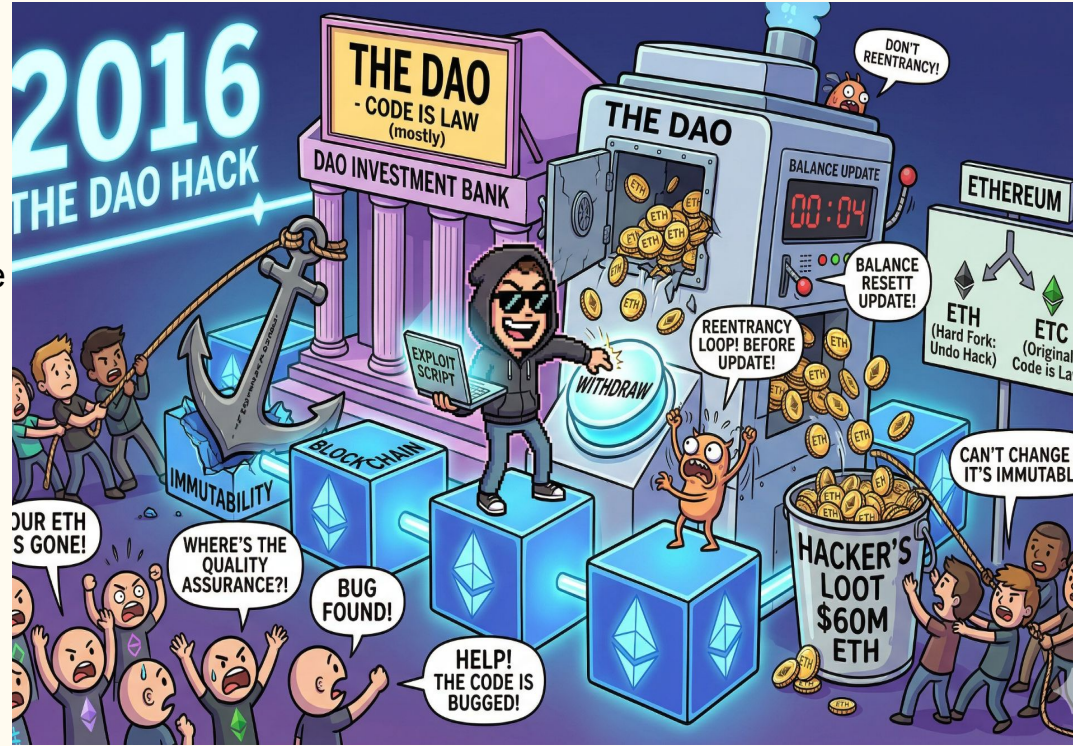
1. **Security as a Quality Pillar:** Security is fundamental to software quality. For smart contracts, quality cannot exist without security.
2. **The Cost of Failure:** Unlike traditional apps where a bug might cause a crash, a smart contract bug leads to direct and permanent financial loss.
3. **Immutability vs. Quality:** Because we cannot "patch" code on the blockchain, SQA must be proactive. Security ensures that the "Quality" remains intact for the entire lifecycle.
4. **Building Trust through SQA:** High-quality SQA processes (like DevSecOps) are essential to ensure users that their assets are safe.

Real-World Consequences: Major Smart Contract Hacks

—

The DAO Hack (2016)

1. **Vulnerability: Reentrancy Bug** (A coding error that allowed an attacker to withdraw funds repeatedly before the balance was updated).
2. **Financial Loss: \$60 Million** worth of Ether was stolen.
3. **Impact:** This was so big that the entire Ethereum blockchain had to be split (Hard Fork) to recover the funds.



The Proposed Adaptive DevSecOps Model

—

The 4 Pillars of the Adaptive Model

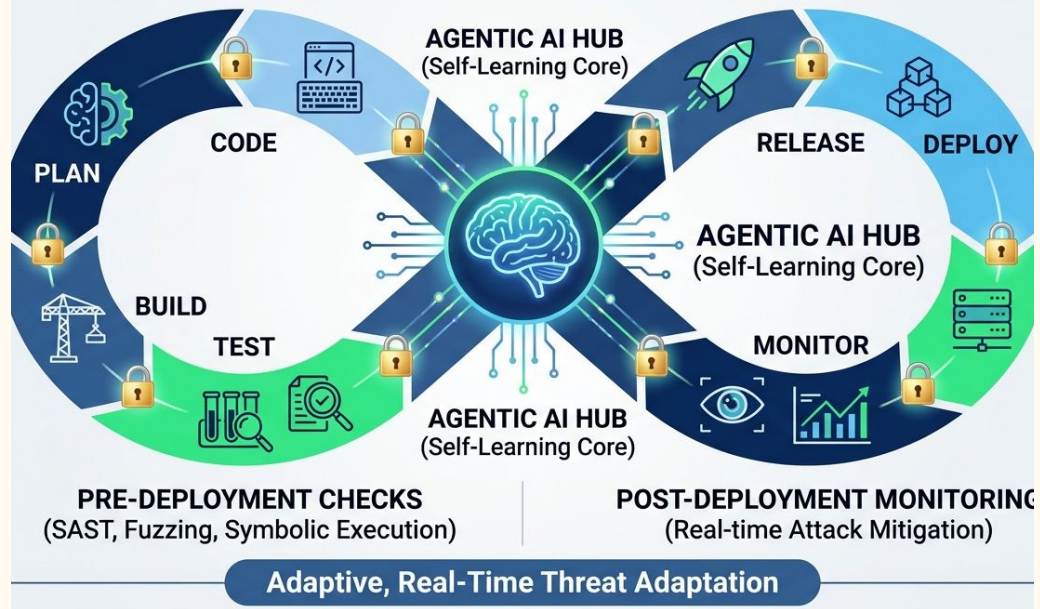
1. Continuous Security (DevSecOps): Security is not just the "final step." It is built into every stage—from writing the code to monitoring it live.

2. Self-Learning AI (Agentic AI): The system doesn't just follow fixed rules; it "learns" from new attack patterns and gets smarter over time.

3. Pre-Deployment Safety (Prevention): Before the contract goes live, the AI scans for known bugs and tests every possible path to find hidden errors.

4. Post-Deployment Guard (Real-Time Protection): After going live, the AI watches the contract 24/7 to detect and block new, unknown attacks instantly.

Adaptive DevSecOps Model for Smart Contract Security



Tools & Technologies Used

—

Traditional Security Tools:

- **Slither & Mythril:** Used for initial static and symbolic analysis to catch common logic errors.
- **Industry Benchmarking:** The model is tested against these tools and **Securify** to prove its superior accuracy.

The AI Engine (LSTM-based SaferSC):

- **Deep Learning:** Uses **LSTM networks** to analyze low-level Opcodes for hidden vulnerabilities.
- **Large-scale Datasets:** Trained on thousands of real Ethereum contracts and historical hack data to learn attack patterns.

The Adaptive Core (Agentic AI Hub):

- **Autonomous Monitoring:** AI agents provide **24/7 protection** and real-time threat detection.
- **Self-Updating Rules:** The system automatically updates security rules based on new attack data without human help.

Development & Automation (CI/CD):

- **CI/CD Pipeline:** Automated using **GitLab/Jenkins** to ensure security is checked during every code update.
- **Environment:** Built and deployed using **Truffle and Hardhat** frameworks.

Performance Comparison

—

Summary of Results:

- 1. The proposed AI model achieved a **0.97 accuracy score** for the most dangerous bugs (Reentrancy).
- 2. It consistently outperformed or matched traditional tools in all categories.
- 3. The AI approach is more effective because it recognizes complex patterns that rule-based tools often miss.

Vulnerability Type	Oyente (Model 1)	Maian (Model 2)	Osiris (Model 3)
Reentrancy (RENT)	0.91	0.94	0.92
Time Dependency (TIMEO)	0.95	0.93	0.97
Arithmetic (ARTHM)	0.93	N/A	N/A

Future Work & Enhancements



Multi-Chain Support: Expanding the model to support Solana and Polkadot.

Self-Healing Contracts: Developing AI that can automatically patch and fix the code.

Conclusion



In conclusion, this research proves that security should not be an afterthought. By using a **DevSecOps approach**, they make security is a core part of the SQA process. This model's **97% accuracy** shows that AI is much more effective than traditional methods at stopping multi-million dollar hacks.